

Angular Interview Q&A Deep Dive

Module-wise Q&A

Section 1: Basics of Signals (Beginner)

1. What are Angular Signals?
 2. Why did Angular introduce Signals?
 3. How do you create a signal?
 4. What is the syntax of reading a signal value?
 5. What is the difference between `signal()` and `computed()`?
 6. What is an `effect()` in Angular Signals?
 7. How do you update a signal value?
 8. What is the difference between `set()` and `update()`?
 9. What is `mutate()` used for?
 10. Are signals synchronous or asynchronous?
 11. What is a `WritableSignal`?
 12. What is a `ReadonlySignal`?
 13. What is the default value of a signal?
 14. Can you store objects/arrays in signals?
 15. What is the purpose of computed signals?
-

Section 2: Intermediate Concepts

16. How do signals achieve fine-grained reactivity?
17. How do computed signals detect dependencies?
18. What is lazy evaluation in computed signals?
19. What is the difference between Signals and `@Input()`?
20. What are Signal Inputs (`input()` API)?
21. What is signal-based output (`output()`)?
22. How do you convert a signal to an observable?
23. How do you convert an observable to a signal?
24. What is the purpose of `toSignal()`?
25. What is the purpose of `toObservable()`?

Angular Interview Q&A Deep Dive

- 26. Can signals replace RxJS completely?
 - 27. How do signals behave inside Angular zoneless apps?
 - 28. What are signal-based forms?
 - 29. What is signal-based routing?
 - 30. What happens if a signal is updated inside another effect?
-

Section 3: Advanced Signal Mechanics

- 31. How does Angular handle circular dependencies in signals?
 - 32. Explain the internal dependency graph of signals.
 - 33. What problems do signals solve compared to Zone.js?
 - 34. How does Angular track dirty nodes in the signal graph?
 - 35. What is the difference between effect() and watch()?
 - 36. What is effect cleanup and why is it needed?
 - 37. How do you stop an effect?
 - 38. How does batching work in signals?
 - 39. What is the difference between push-based and pull-based reactivity?
 - 40. What reactivity model do signals follow?
 - 41. Can signal updates cause infinite loops?
 - 42. How does Angular ensure deterministic reactivity order?
 - 43. What is the cost of creating too many signals?
 - 44. What are limitations of signals?
-

Section 4: Real-World Usage

- 45. How do signals improve performance in large applications?
 - 46. Why are signals better for component state than RxJS?
 - 47. How do you create a store using signals?
 - 48. What is the recommended folder structure for signal-based stores?
 - 49. How would you build CRUD operations using signals?
 - 50. How do you handle pagination using signals?
 - 51. How do you handle dependent dropdowns using signals?
 - 52. How do signals help reduce unnecessary change detection?
-

Angular Interview Q&A Deep Dive

- 53. How to use signals in services?
 - 54. Can signals be shared across modules?
 - 55. How do signals behave during component destruction?
-

Section 5: Architecture, Patterns & Best Practices

- 56. What is the Signal Store Pattern?
- 57. How does Angular's @ngrx/signals differ from normal signals?
- 58. Should you store API data in signals or observables?
- 59. What state should NOT be stored in signals?
- 60. How do signals help build reactive UIs without Zone.js?
- 61. What is fine-grained change detection?
- 62. How do you debug signal updates?
- 63. What are common mistakes when using signals?

Signals Module:

1 What are Angular Signals?

Answer:

Signals are a new reactivity model introduced in Angular 16–17. A **signal is a reactive value that notifies Angular when it changes**, allowing the UI to update automatically without change detection cycles.

2 Why did Angular introduce Signals?

Answer:

Angular introduced signals to:

- Replace overly complex **Zone.js-based change detection**
- Provide **fine-grained reactivity**
- Improve **performance** by updating only the affected components
- Align Angular with modern reactive systems (like SolidJS, Vue, React)

Signals fix Angular's old issues:

- Unpredictable change detection cycles
 - Re-renders happening even when your component doesn't need updates
 - Difficulty debugging reactivity
-

Angular Interview Q&A Deep Dive

3 How do Signals work internally?

Answer:

Internally, a signal maintains:

- A **value store**
- A list of **dependents** (computeds/effects)
- A **subscriber graph**

When you update a signal:

1. Its internal value changes
2. Angular marks dependents as “dirty”
3. Only the affected parts re-render

This is called **fine-grained reactivity**.

4 What are signal(), computed(), and effect()?

API	Purpose
signal()	Holds a reactive value
computed()	Creates derived values (auto recalculated)
effect()	Runs side effects when signals change

Example:

```
count = signal(1);  
double = computed(() => count() * 2);  
effect(() => console.log(double()));
```

5 Explain synchronous nature of Signals.

Answer:

Signals are **fully synchronous**:

- When you update a signal, all dependents (computed and effects) run **immediately**.
 - No microtasks, no async scheduling.
 - Predictable order of execution.
-

6 How are Signals different from RxJS Observables?

Angular Interview Q&A Deep Dive

Signals	Observables
Synchronous	Asynchronous + Streams
Holds one value at a time	Holds multiple events over time
Simple	Powerful but complex
Less boilerplate	Requires subscriptions, unsubscribe, operators
UI reactivity	Data streams, APIs, events

Use **signals** for UI state, **observables** for backend streams.

7 What is set(), update(), and mutate()?

✓ set()

Replaces the entire value.

```
count.set(10);
```

✓ update()

Recomputes value based on previous.

```
count.update(v => v + 1);
```

✓ mutate()

Used for objects & arrays.

```
items.mutate(arr => arr.push("New"));
```

8 What are Signal-Based Inputs in Angular?

Answer:

Instead of @Input(), Angular now supports:

```
@Component({  
  selector: 'app-card',  
  inputs: {  
    data: input.required<string>()  
  }  
})
```

Angular Interview Q&A Deep Dive

```
export class Card { }
```

These are **signals**, not old @Inputs.

9 What is WritableSignal vs ReadonlySignal?

Answer:

Type	Description
WritableSignal	Can change value (set, update, mutate)
ReadonlySignal	Read-only (exposed to child components)

10 What is “effect cleanup”?

Answer:

Each effect() can return a cleanup function:

```
effect(onCleanup => {  
  const timer = setInterval(...);  
  
  onCleanup(() => clearInterval(timer));  
});
```

Runs automatically before the next execution.

1 1 Can you use Signals without Zone.js?

Answer:

Yes!

Angular 18 allows **full zoneless apps** because signals manage change detection internally.

1 2 How do Signals improve performance?

Answer:

- No global change detection
 - No digest cycles
 - No runEverywhere() effect
 - Only the affected UI recomputes
-

Angular Interview Q&A Deep Dive

- Component renders become predictable
-

1 3 What is fine-grained reactivity?

Answer:

Only the **exact piece of UI** that depends on a signal re-renders, not the entire component tree.

1 4 Signals vs. ChangeDetectionStrategy.OnPush?

Answer:

- OnPush is still useful but not required.
 - Signals automatically give **OnPush-level performance**, even better.
-

1 5 Are Signals replacing RxJS?

Answer:

No.

Signals replace UI state handling.

RxJS remains for:

- API calls
- WebSockets
- Streams
- Complex async flows

Use both together.

1. What are Angular Signals?

Signals are **reactive variables** that automatically notify Angular when their value changes, without using Zone.js.

```
count = signal(0);
```

2. Why did Angular introduce Signals?

To solve problems with:

- ✓ Unnecessary change detection
 - ✓ Zone.js overhead
 - ✓ Too much RxJS usage for simple state
 - ✓ To enable fine-grained reactivity (update only what changes)
-

Angular Interview Q&A Deep Dive

3. How do you create a signal?

```
name = signal("Rahul");
```

4. How do you read a signal?

```
console.log(this.name()); // call as a function
```

5. Difference between signal() and computed()

signal()	computed()
Holds a value	Derives value from other signals
Writable	Read-only
Updates manually	Updates automatically

6. What is an effect()??

Runs a side-effect whenever a signal changes.

```
effect(() => console.log(this.count()));
```

7. How do you update a signal value?

```
count.set(5);
```

8. Difference between set() and update()

- **set(value)** → replaces value
- **update(fn)** → modifies existing value

```
count.update(c => c + 1);
```

9. What is mutate() used for?

Used for updating **objects/arrays** inside signals.

```
students.mutate(s => s.push({ name: "Aisha" }));
```

10. Are signals synchronous or asynchronous?

Signals are **100% synchronous** — updates happen instantly.

Angular Interview Q&A Deep Dive

11. What is WritableSignal?

Returned by signal().

We can **set**, **update**, **mutate** it.

12. What is ReadonlySignal?

Returned by:

- computed()
- asReadonly()

It **cannot** be changed manually.

13. What is the default value of a signal?

It is **whatever value you pass**:

```
signal(0); // default = 0
```

14. Can you store objects/arrays in signals?

YES.

```
user = signal({ name: "Rahul", age: 25 });
```

15. What is the purpose of computed signals?

To automatically compute a value based on other signals.

SECTION 2 — INTERMEDIATE (16–30)

16. How do signals achieve fine-grained reactivity?

Signals update **only** the component/template that reads them — not the whole app.

17. How do computed signals detect dependencies?

Angular automatically tracks **any signal read** inside the computed function.

18. What is lazy evaluation in computed signals?

Computed only re-evaluates **when needed**, not on every update.

Angular Interview Q&A Deep Dive

19. Difference between Signals and @Input()?

@Input	Signal
One-way parent → child	Reactive and auto-updating
Static binding	Computed & reactive features
No built-in reactivity	Built-in reactivity

20. What are signal inputs (input() API)?

`myData = input<number>();`

A reactive input variable.

21. What is signal-based output (output())

A replacement for EventEmitter.

`submitted = output<string>();`

22. Convert signal to observable?

`toObservable(this.count);`

23. Convert observable to signal?

`toSignal(user$);`

24. Purpose of toSignal()

Converts async observable streams into synchronous signals.

25. Purpose of toObservable()

To use signals with RxJS operators.

26. Can signals replace RxJS completely?

NO.

RxJS still required for:

- ✓ HTTP streams
 - ✓ WebSockets
-

Angular Interview Q&A Deep Dive

✓ Debounce, Retry, MergeMap

✓ Async event streams

27. How do signals behave in zoneless Angular?

Perfectly — signals remove the need for Zone.js.

28. What are signal-based forms?

Reactive forms where valueChanges become signals.

29. What is signal-based routing?

Router exposes signals:

- route.params
 - route.queryParams
-

30. What happens if a signal is updated inside an effect?

It may cause **infinite loops** → must avoid.

SECTION 3 — ADVANCED (31–44)

31. How does Angular handle circular dependencies in signals?

Angular detects circular graphs and throws an error.

32. Explain internal dependency graph.

Signals form a **DAG (Directed Acyclic Graph)** → no cycles allowed.

33. What problems do signals solve compared to Zone.js?

- ✓ No global change detection
 - ✓ Only affected components re-render
 - ✓ Better performance
 - ✓ Predictable updates
-

34. How does Angular track dirty nodes?

Angular Interview Q&A Deep Dive

Each signal update marks itself and dependent computed signals as **dirty**, then recomputes only those parts.

35. Difference between `effect()` and `watch()`?

- **`effect()`** → automatic
 - **`watch()`** → used for debugging; manually controlled
-

36. What is effect cleanup?

Returning a cleanup function inside an effect.

37. How to stop / destroy an effect?

Use:

```
const stop = effect(...);
```

```
stop();
```

38. What is batching in signals?

Multiple updates are combined into a single re-render.

39. Difference between push vs pull reactivity

- **Signals = Pull-based** — template pulls values
 - **Observables = Push-based** — values pushed over time
-

40. What is the reactivity model of signals?

Synchronous pull-based graph reactivity.

41. Can signal updates cause infinite loops?

YES, if a signal updates itself inside an effect.

42. How does Angular ensure deterministic reactivity?

Evaluates signals in topological sorted order.

Angular Interview Q&A Deep Dive

43. Cost of creating too many signals?

Low memory cost, but too many effects = performance issues.

44. Limitations of signals

- No async behavior
 - Cannot replace RxJS
 - No built-in debouncing or throttling
 - Circular computations not allowed
-

SECTION 4 — REAL WORLD (45–55)

45. How do signals improve performance in large apps?

Only affected component bindings update — not entire tree.

46. Why are signals better than RxJS for component state?

Simpler, synchronous, no subscriptions required.

47. How do you create a store using signals?

Use a service with signals:

```
@Injectable()
class CounterStore {
  count = signal(0);
  increment = () => this.count.update(c => c + 1);
}
```

48. Folder structure for signal stores

```
/stores
  /user.store.ts
  /cart.store.ts
```

Angular Interview Q&A Deep Dive

49. CRUD operations using signals?

Example Add:

```
items.update(list => [...list, newItem]);
```

50. Pagination using signals

Page, limit, total → all stored in signals.

51. Dependent dropdown using signals

Second dropdown is a computed signal based on first dropdown.

52. How do signals reduce change detection?

Works without zones → no global re-render.

53. Use signals in services?

VERY COMMON — signals replace BehaviorSubject.

54. Can signals be shared across modules?

Yes, via services or stores.

55. How do signals behave on component destroy?

Effects auto-cleanup with ownership tracking.

SECTION 5 — ARCHITECTURE & BEST PRACTICES (56–63)

56. What is the Signal Store Pattern?

A global state management pattern built using signals.

57. Difference between Angular Signal Store and normal signals?

Signal Store adds:

- ✓ Actions
 - ✓ Selectors
-

Angular Interview Q&A Deep Dive

✓ State immutability

✓ Dev tools

58. Should you store API data in signals or observables?

- API Streams → Observables
- UI/View State → Signals

59. What state should NOT be stored in signals?

- Async streams
- WebSocket data
- Realtime events

60. How do signals help in Angular without Zone.js?

Only re-renders what changed.

No dirty-checking for entire components.

61. What is fine-grained change detection?

Update only the exact DOM nodes that depend on the changed signal.

62. How do you debug signals?

Use watch()

Use Chrome Angular DevTools (v18+)

63. Common mistakes using signals

- ✗ Updating signals inside effects
- ✗ Storing async data
- ✗ Using many nested computed values
- ✗ Mutating directly without mutate()

Angular Interview Q&A Deep Dive

Component Module

◆ 1. What is a Component in Angular?

A component is a basic UI building block in Angular. It controls a part of the screen called a *view*. It includes:

- **TypeScript class**
 - **HTML template**
 - **CSS styles**
 - **Component metadata (@Component)**
-

◆ 2. What are the main parts of a Component?

- **Decorator** → @Component({...})
 - **Template** → HTML
 - **Class** → business logic
 - **Styles** → CSS/SCSS
-

◆ 3. What is a Module in Angular?

A module is a container that groups related components, directives, pipes, and services. Example: AppModule.

◆ 4. Difference between Component and Module

Component	Module
Controls UI	Groups components
Only one template	No template
Must belong to a module	Root level container
Represents view	Represents feature

◆ 5. What is @NgModule?

A decorator that defines a module's metadata like:

- declarations
 - imports
 - providers
-

Angular Interview Q&A Deep Dive

- bootstrap

◆ 6. What are declarations in a module?

Components, Directives, Pipes → must be declared before using.

◆ 7. What are imports in a module?

External modules required by the current module.

◆ 8. What are providers in a module?

Services that will be dependency-injected.

◆ 9. What is bootstrap in NgModule?

Root component that Angular loads first.

◆ 10. What is a Feature Module?

A module created to organize specific functionality (example: UserModule, AdminModule).

◆ 11. What is a Shared Module?

A module containing reusable components, directives, pipes to be used across the app.

◆ 12. What is a Core Module?

Contains singleton services and things loaded once (AuthService, HTTP Interceptor).

◆ 13. What is Lazy Loading?

Loading modules on demand to improve performance.

◆ 14. How do you create a component?

ng generate component componentName

Angular Interview Q&A Deep Dive

◆ 15. How do you create a module?

ng generate module moduleName

◆ 16. What is a Root Module?

The main application module → AppModule.

◆ 17. Can a component be declared in two modules?

✗ No. A component can belong to only one module.

◆ 18. What is a Standalone Component?

Component without NgModule (Angular 14+).

Uses:

```
@Component({  
  standalone: true  
})
```

◆ 19. What is View Encapsulation?

Controls how CSS is applied:

- ShadowDom
 - Emulated (default)
 - None
-

◆ 20. Explain Component Lifecycle Hooks

- ngOnInit()
 - ngOnChanges()
 - ngDoCheck()
 - ngAfterViewInit()
 - ngAfterContentInit()
 - ngOnDestroy()
-

Angular Interview Q&A Deep Dive

◆ 21. What is `ngOnInit` used for?

Initialization logic like API calls.

◆ 22. What is `ngOnDestroy` used for?

Unsubscribe Observables, remove event listeners.

◆ 23. Communication between components?

- `Input()`
 - `Output()`
 - `ViewChild`
 - Service with Subject
-

◆ 24. What is `@Input`?

Used to pass data from parent to child component.

◆ 25. What is `@Output`?

Used to emit events from child to parent.

◆ 26. What is `ViewChild`?

Access child component, DOM element, or directive.

◆ 27. What is `ContentChild`?

Access projected content using `<ng-content>`.

◆ 28. What is `ng-content`?

Used for content projection like slots.

◆ 29. What is Change Detection?

Angular checks and updates the UI based on data changes.

Angular Interview Q&A Deep Dive

◆ 30. Change Detection Strategies

- Default
 - OnPush (performance)
-

◆ 31. What is a Pipe?

Used to transform data in templates.

◆ 32. What is a Directive?

Adds behavior to DOM elements.

Types:

- Structural → *ngIf, *ngFor
 - Attribute → ngClass, ngStyle
-

◆ 33. What is a service?

Class providing logic/data using Dependency Injection.

◆ 34. What is Dependency Injection?

Angular automatically provides required services to components.

◆ 35. What is Module Federation?

Sharing modules between different Angular applications (microfrontends).

◆ 36. What is the purpose of AppComponent?

Main UI component loaded first.

◆ 37. What is RouterModule?

Module responsible for routing.

Angular Interview Q&A Deep Dive

◆ 38. What is a Routing Module?

A separate module used only for routes:
app-routing.module.ts

◆ 39. Can a module import itself?

✗ No, it creates a circular dependency.

◆ 40. What are entryComponents? (Deprecated)

Earlier used for dynamic component loading (Angular 8-). Now not needed.

◆ 41. What is Ahead-of-Time (AOT) compilation?

Compiles code at build time → faster performance.

◆ 42. What is Just-in-Time (JIT) compilation?

Compilation at runtime (during browser load).

◆ 43. What are Angular Schematics?

CLI instructions to generate files (components, modules, services).

◆ 44. What is a Smart Component?

Handles data & logic.

◆ 45. What is a Dumb Component?

UI only; receives data via @Input.

◆ 46. What is a HostListener?

Listens to events on host element.

◆ 47. What is HostBinding?

Binds properties to host element.

Angular Interview Q&A Deep Dive

◆ 48. What is a Module Import Cycle Error?

Occurs when module A imports B and B imports A.

◆ 49. What is a dynamic component?

A component loaded at runtime using ComponentFactoryResolver.

◆ 50. Why modular architecture in Angular?

Benefits:

- Maintainability
- Reusability
- Lazy loading
- Clean structure

Angular Interview Q&A Deep Dive

Services & Dependency Injection Module

◆ 1. What is a Service in Angular?

A **service** is a class used to share logic, data, and functions across components.

◆ 2. Why do we use services?

- Code reusability
 - Centralized business logic
 - Communication between components
 - API calls & data storage
-

◆ 3. How do you create a service?

ng generate service user

◆ 4. What is Dependency Injection (DI)?

DI is a design pattern where objects receive dependencies rather than creating them manually.

◆ 5. How do you inject a service into a component?

```
constructor(private userService: UserService) {}
```

◆ 6. What is @Injectable()?

Decorator that marks a class as available for injection.

◆ 7. Explain providedIn: 'root'

- Registers service in the **root injector**
 - Makes it a **singleton**
 - No need to add in providers array
-

◆ 8. What is a Singleton Service?

A single instance shared across entire application.

Angular Interview Q&A Deep Dive

◆ 9. How do you provide a service only in a specific module?

Add service in module's providers array.

◆ 10. What is hierarchical dependency injection?

Angular injectors form a hierarchy:

- Root injector
- Module injector
- Component injector

Lower injectors can override services of parent injectors.

◆ 11. What is a root injector?

Top-level injector created when Angular app starts.

◆ 12. What happens when providedIn is 'any'?

Creates a **separate instance of the service** for every lazy-loaded module.

◆ 13. Difference: providedIn 'root' vs 'any'

root	any
Singleton	Multiple instances
Best for global services	Best for lazy modules

◆ 14. What is providedIn: 'platform'?

Service is shared across multiple Angular apps running on the same page.

◆ 15. What is a service scope?

Determines how long a service instance lives and where it can be injected.

◆ 16. What is dependency injection token?

A **key** used to lookup a dependency in the injector.

Angular Interview Q&A Deep Dive

◆ 17. What is @Inject decorator?

Explicitly injects value providers or tokens.

◆ 18. What are Angular provider types?

- Class provider
 - Value provider
 - Factory provider
 - Existing provider
-

◆ 19. Example of value provider

```
providers: [  
  { provide: 'API_URL', useValue: 'https://test.com' }  
]
```

◆ 20. Example of factory provider

```
providers: [  
  { provide: LogService, useFactory: () => new LogService(true) }  
]
```

◆ 21. Example of existing provider

```
providers: [  
  { provide: Logger, useExisting: LogService }  
]
```

◆ 22. What is multi-provider?

Allows multiple values for the same token using multi: true.

◆ 23. What is a service used for API calls?

An **HTTP service** using HttpClient.

Angular Interview Q&A Deep Dive

◆ 24. Why put API calls in services?

- Separation of concerns
 - Reusable
 - Testable
 - Easy to mock
-

◆ 25. What is HttpClientModule?

Module that enables HttpClient functionalities.

◆ 26. What is HttpInterceptor?

Special service to modify outgoing requests/response globally.

Example uses:

- Add JWT tokens
 - Logging
 - Error handling
-

◆ 27. What is a tree-shakeable provider?

Service removed from bundle if not used (providedIn supports this).

◆ 28. How to disable tree-shaking?

Provide service inside providers[].

◆ 29. What is a shared service for communication?

Service using RxJS Subject/BehaviorSubject to pass data between components.

◆ 30. Example of BehaviorSubject service

```
data$ = new BehaviorSubject<string>('default');
```

◆ 31. What is a Subject vs BehaviorSubject?

- **Subject:** No initial value
-

Angular Interview Q&A Deep Dive

- **BehaviorSubject:** Requires initial value & stores latest value

◆ 32. What is DI in constructor?

Angular automatically supplies needed services when creating components.

◆ 33. Can Angular inject private/protected services?

Yes — visibility does not affect DI.

◆ 34. What happens if service is not provided?

Angular throws:

✗ *NullInjectorError: No provider for service*

◆ 35. How to create a component-level service?

```
@Component({  
  providers: [UserService]  
})
```

Creates a new instance *per component*.

◆ 36. Difference between module-level and component-level services

Module Provider	Component Provider
Singleton	New instance per component
Shared	Isolated

◆ 37. What is InjectionToken?

Used for non-class dependencies.

```
export const API_TOKEN = new InjectionToken<string>('API_TOKEN');
```

◆ 38. What is optional dependency?

You can mark a dependency as optional:

```
constructor(@Optional() private logger?: LoggerService) {}
```

Angular Interview Q&A Deep Dive

◆ 39. What is self-injection?

Only search current injector:

```
constructor(@Self() service: A) {}
```

◆ 40. What is skipSelf injection?

Skip current injector and search parent:

```
constructor(@SkipSelf() service: A) {}
```

◆ 41. What is host injection?

Looks only at host element's injector.

◆ 42. What is forwardRef?

Used to refer to classes that are not yet defined.

◆ 43. What is circular dependency in DI?

Service A depends on B and B depends on A → error.

◆ 44. How to avoid circular dependency?

- Use interfaces
 - Break logic
 - Use factory providers
 - Use InjectionToken
-

◆ 45. What is providedIn: 'root' default behavior?

Creates 1 instance throughout the app.

◆ 46. Why use DI instead of new keyword?

- Easier testing
 - Automatic lifecycle
 - Cleaner code
-

Angular Interview Q&A Deep Dive

- Global sharing
-

◆ 47. Can DI be used outside Angular classes?

Yes, in standalone environment using inject().

◆ 48. What is the inject() function?

Angular 14+ way to inject services without constructor.

```
const logger = inject(LoggerService);
```

◆ 49. What is environment injector?

New DI container (Angular 14+) used in standalone apps.

◆ 50. Can services depend on other services?

Yes — multi-level nested injection is supported.

◆ 51. Can DI inject values from environment.ts?

Yes using value providers.

◆ 52. What is the lifetime of a service?

Depends on where it is provided:

- Root → lifetime of app
- Module → lifetime of that module
- Component → lifetime of component

Angular Interview Q&A Deep Dive

Data Binding Module

◆ 1. What is Data Binding in Angular?

Data binding is the technique to **connect component data with the UI** and vice-versa.

◆ 2. How many types of data binding are in Angular?

1. **Interpolation** → {{ }}
 2. **Property Binding** → [property]
 3. **Event Binding** → (event)
 4. **Two-way Binding** → [(ngModel)]
-

◆ 3. What is Interpolation?

Used to display component values in UI.

```
<h1>{{ title }}</h1>
```

◆ 4. Can interpolation evaluate expressions?

Yes — simple expressions only.

```
{{ 5 + 5 }}
```

```
{{ user.name.toUpperCase() }}
```

◆ 5. Can interpolation call functions?

Yes, but not recommended (performance cost).

◆ 6. What is Property Binding?

Used to set DOM element or child component properties.

```
<img [src]="imageUrl">
```

◆ 7. Difference: Interpolation vs Property Binding

Interpolation	Property Binding
Works only with strings	Works with all data types
Syntax: {{ }}	Syntax: []

Angular Interview Q&A Deep Dive

Value appears inside text content	Bind to DOM property
-----------------------------------	----------------------

◆ 8. What is Attribute Binding?

Used for HTML attributes (not DOM properties).

```
<td [attr.colspan]="2"></td>
```

◆ 9. What is Class Binding?

```
<div [class.active]="isActive"></div>
```

◆ 10. What is Style Binding?

```
<div [style.color]="isRed ? 'red' : 'blue'"></div>
```

◆ 11. What is Event Binding?

Used to listen to events from DOM.

```
<button (click)="handleClick()">Click</button>
```

◆ 12. What is \$event in Angular?

Special variable that contains event data.

```
<input (input)="onInput($event)">
```

◆ 13. What is Two-way Binding?

Combination of:

- Property Binding ([])
- Event Binding (())

Uses [(ngModel)].

```
<input [(ngModel)]="name">
```

◆ 14. Which module is required for ngModel?

FormsModule

Angular Interview Q&A Deep Dive

◆ 15. What is the banana-in-a-box syntax?

Two-way binding syntax: [(ngModel)]

◆ 16. What is Data Flow in One-Way Binding?

Component → Template

OR

Template → Component

Not both.

◆ 17. What is Data Flow in Two-Way Binding?

Component ⇌ Template

(both directions)

◆ 18. Why avoid two-way binding sometimes?

- Harder to debug
 - More coupling
 - Performance cost
-

◆ 19. How to achieve two-way binding manually?

```
<input [value]="name" (input)="name = $event.target.value">
```

◆ 20. What happens if ngModel is used without FormsModule?

Angular throws error:

✗ *"Can't bind to ngModel since it isn't a known property of input."*

◆ 21. Can we bind to custom component properties?

Yes.

```
<user-card [user]="selectedUser"></user-card>
```

◆ 22. What is @Input()?

Used to receive data from parent.

Angular Interview Q&A Deep Dive

◆ 23. What is @Output()?

Used to send events/data to parent.

◆ 24. Example of @Output with EventEmitter

```
@Output() sendData = new EventEmitter<string>();
```

◆ 25. What is EventEmitter?

A class that emits custom events.

◆ 26. Difference @Input vs Property Binding

@Input

Property Binding

Defines a property to receive data

Sends data from parent to child

◆ 27. Can a component use both Input and Output?

Yes — common for reusable components.

◆ 28. What is Change Detection?

Angular system that updates the UI when data changes.

◆ 29. How does Angular detect data changes?

Using Zones (Zone.js) or Signals (Angular 18+).

◆ 30. What is Template Reference Variable?

A variable for accessing DOM elements.

```
<input #box>
```

◆ 31. How to use template variable in event binding?

```
<button (click)="log(box.value)">Log</button>
```

◆ 32. Can we use property & event binding on same element?

Angular Interview Q&A Deep Dive

Yes.

◆ 33. What is Safe Navigation Operator?

Prevents runtime errors when value is null/undefined.

```
{{ user?.name }}
```

◆ 34. What is ngIf?

Structural directive for conditionally rendering elements.

```
<div *ngIf="isLoggedIn">Welcome</div>
```

◆ *35. Difference between [hidden] and ngIf

[hidden]

*ngIf

Hides but keeps in DOM Removes from DOM

Faster

Safer on performance

◆ 36. What is ngFor?

Used to loop over collections.

```
<li *ngFor="let item of items">{{ item }}</li>
```

◆ 37. What is trackBy?

Improves performance for ngFor lists.

◆ 38. Example of trackBy

```
trackById(index, item) { return item.id; }
```

◆ 39. What is property vs attribute?

Property	Attribute
JS runtime value	HTML static value
Can change	Fixed

Angular Interview Q&A Deep Dive

◆ 40. Why property binding is preferred?

Because DOM properties reflect real-time values.

◆ 41. What is Input aliasing?

@Input('itemData') data: any;

Usage:

```
<card [itemData]="value"></card>
```

◆ 42. What is Output aliasing?

@Output('onSave') saveEvent = new EventEmitter();

◆ 43. Example of two-way binding in custom component

Inside child:

@Input() value: string = '';

@Output() valueChange = new EventEmitter<string>();

Usage:

```
<my-input [(value)]="name"></my-input>
```

◆ 44. Can we bind styles dynamically?

Yes.

```
<div [style.background]="color"></div>
```

◆ 45. Can we bind classes dynamically?

Yes.

```
<div [class.error]="isError"></div>
```

◆ 46. What is text interpolation used for?

Displaying component data in UI.

◆ 47. Can interpolation bind HTML?

Angular Interview Q&A Deep Dive

No — it escapes HTML for security.

Use [innerHTML] instead.

◆ 48. What is event bubbling in Angular?

Events propagate upward from child to parent.

◆ 49. Can Angular stop event bubbling?

Yes.

```
<button (click)="$event.stopPropagation()">Stop</button>
```

◆ 50. What happens when you bind to a disabled property?

It enables/disables the DOM element dynamically.

◆ 51. Can we bind to multiple classes?

Yes.

```
<div [ngClass]="{ active: isActive, error: hasError }"></div>
```

◆ 52. What is ngStyle?

Dynamic inline styling.

```
<div [ngStyle]="{ color: textColor }"></div>
```

Angular Interview Q&A Deep Dive

Pipes Module:

◆ 1. What is a Pipe in Angular?

A pipe is a feature that transforms data before displaying it in the view.

◆ 2. Example of using a pipe

```
{{ message | uppercase }}
```

◆ 3. Why are pipes used?

- Format data
 - Reduce template logic
 - Improve readability
-

◆ 4. Types of Pipes in Angular

1. Built-in pipes
 2. Custom pipes
-

◆ 5. Examples of built-in pipes

- uppercase
 - lowercase
 - date
 - currency
 - percent
 - json
 - slice
 - async
-

◆ 6. What is a Custom Pipe?

A user-created pipe for custom transformation logic.

Angular Interview Q&A Deep Dive

◆ 7. How to create a pipe?

ng generate pipe myCustom

◆ 8. Pipe Naming Convention

Pipe class must end with Pipe and filename with .pipe.ts.

◆ 9. What is the @Pipe decorator?

Defines metadata for a pipe.

◆ 10. Syntax of Pipe decorator

```
@Pipe({  
  name: 'myPipe'  
})
```

◆ 11. What is transform() method?

Every pipe must define this method to transform data.

◆ 12. Example of Custom Pipe

```
transform(value: number) {  
  return value * 2;  
}
```

◆ 13. Can pipes accept parameters?

Yes.

```
{{ amount | currency:'INR' }}
```

◆ 14. Multiple parameters in pipes

```
{{ date | date:'dd/MM/yyyy' }}
```

Angular Interview Q&A Deep Dive

◆ 15. Can we use multiple pipes in a single expression?

Yes.

```
{{ name | uppercase | slice:0:3 }}
```

◆ 16. What is a Pure Pipe?

Calculated only when input value changes.

◆ 17. What is an Impure Pipe?

Calculated on every change detection cycle.

◆ 18. How to make a pipe impure?

```
@Pipe({  
  name: 'myPipe',  
  pure: false  
})
```

◆ 19. When to use an Impure Pipe?

When the input is:

- Array
 - Object
 - Or frequently changing data
-

◆ 20. Why avoid impure pipes?

They run very frequently → can cause performance issues.

◆ 21. What is the Async Pipe?

Automatically subscribes to Observables/Promises and updates UI.

◆ 22. Example Async Pipe

```
{{ user$ | async }}
```

Angular Interview Q&A Deep Dive

◆ 23. What happens when Observable emits new value?

Async pipe updates template automatically.

◆ 24. Does Async Pipe handle unsubscribe?

Yes — it unsubscribes automatically.

◆ 25. What is Date Pipe?

Formats date according to locale.

◆ 26. Example Date Pipe

```
{{ today | date:'short' }}
```

◆ 27. Common Date Pipe formats

- short
 - medium
 - long
 - fullDate
-

◆ 28. What is Percent Pipe?

Formats number as percentage.

```
{{ 0.34 | percent }}
```

◆ 29. What is Number Pipe?

Formats numbers.

```
{{ 12345 | number:'1.0-2' }}
```

◆ 30. What is Currency Pipe?

Formats monetary values.

```
{{ 1500 | currency:'USD' }}
```

Angular Interview Q&A Deep Dive

◆ 31. Can we chain async with other pipes?

Yes.

```
{{ user$ | async | json }}
```

◆ 32. What is Json Pipe?

Converts objects to JSON string (for debugging).

◆ 33. What is Slice Pipe?

Extracts a portion of array or string.

```
{{ 'Hello' | slice:1:4 }}
```

◆ 34. What is TitleCase Pipe?

Capitalizes the first letter of each word.

◆ 35. How to use pipe inside an HTML attribute?

```
<input [value]="name | uppercase">
```

◆ 36. Can pipes be used in component.ts?

Not directly — need to inject pipe service manually.

◆ 37. Example using pipe in TypeScript

```
constructor(private title: TitleCasePipe) {}
```

◆ 38. Which Angular module contains pipes?

Most built-in pipes come from CommonModule.

◆ 39. Can pipes be asynchronous?

Only async pipe is asynchronous.

Angular Interview Q&A Deep Dive

◆ 40. What is a pipe transform return type?

It can return:

- string
 - number
 - boolean
 - object
 - array
 - any transformed value
-

◆ 41. What is pure: true?

Pipe recalculates only when input reference changes.

◆ 42. Example of pure pipe (default)

```
@Pipe({ name: 'purePipe' })
```

◆ 43. When does pure pipe NOT recalculate?

When:

```
arr.push(value)
```

Because reference DID NOT change.

◆ 44. How to fix pipe not updating array changes?

Use a new array:

```
this.arr = [...this.arr];
```

◆ 45. Can pipes modify original data?

No — pipes should be pure functions that return new values.

◆ 46. Can we pass object to pipe?

Yes.

```
{{ user | myPipe:extraData }}
```

Angular Interview Q&A Deep Dive

◆ 47. What are pipe arguments called?

Parameters.

◆ 48. Can pipe arguments be dynamic?

Yes.

```
{{ price | currency:currencyCode }}
```

◆ 49. Example of pipe returning HTML

Use careful sanitization.

◆ 50. Why avoid calling functions inside pipes?

Pipes may be called multiple times → expensive operations hurt performance.

◆ 51. What is a decorator used for pipe definition?

@Pipe()

◆ 52. Can we debug pipe transform logic?

Yes — console log inside transform, but only for dev use.

Angular Interview Q&A Deep Dive

Routing Module

1. What is Angular Routing?

Routing in Angular allows you to **navigate between different views (components)** based on the URL without reloading the entire page.

2. Why do we use Routing in Angular?

- To build **SPA (Single Page Applications)**
 - Faster navigation
 - Avoid full-page reload
 - Better user experience
-

3. What is a Route?

A route is a **mapping between a URL path and a component**.

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent }  
];
```

4. What is RouterModule? Why is it required?

RouterModule contains all the router services and directives.

It is required to enable routing in an Angular app.

```
imports: [RouterModule.forRoot(routes)]
```

5. Difference between forRoot() and forChild()?

forRoot()	forChild()
Used in AppModule	Used in feature modules
Registers router service	Does not register extra router service
Called once only	Can be called many times

6. What is RouterOutlet?

A directive that **displays the component** associated with the current route.

```
<router-outlet></router-outlet>
```

Angular Interview Q&A Deep Dive

7. What is RouterLink?

A directive for navigation without page reload.

```
<a routerLink="/dashboard">Dashboard</a>
```

8. Difference between routerLink & href?

routerLink	href
SPA navigation	Reloads full page
Controlled by Angular router	Controlled by browser
Faster	Slower

9. How to pass parameters to a route?

Route config:

```
{ path: 'user/:id', component: UserComponent }
```

Navigate:

```
<a [routerLink]="['/user', 10]">User 10</a>
```

10. How to read route parameters?

```
this.route.paramMap.subscribe(params => {  
  this.id = params.get('id');  
});
```

11. What is Query Parameter?

Example URL:

```
/products?category=mobile
```

Read query param:

```
this.route.queryParamMap.subscribe(q => {  
  this.category = q.get('category');  
});
```

Angular Interview Q&A Deep Dive

12. How to navigate programmatically?

```
this.router.navigate(['/home']);
```

With params:

```
this.router.navigate(['/user', 5]);
```

13. What are Nested Routes?

Allows routes inside another component.

```
{
  path: 'admin',
  component: AdminComponent,
  children: [
    { path: 'users', component: UsersComponent }
  ]
}
```

14. What is Lazy Loading in Angular?

Lazy loading loads a module **only when the user navigates to it**, reducing the initial bundle size.

15. How to configure lazy loading?

app-routing.module.ts:

```
{
  path: 'products',
  loadChildren: () => import('./products/product.module').then(m => m.ProductModule)
}
```

16. What are Route Guards?

Guards protect routes from unauthorized or unwanted access.

Types:

1. CanActivate
 2. CanActivateChild
 3. CanDeactivate
-

Angular Interview Q&A Deep Dive

4. Resolve
5. CanLoad
6. CanMatch (Angular 15+)

17. What is CanActivate?

Prevents navigation to a route.

```
canActivate(): boolean { ... }
```

18. What is CanDeactivate?

Used to **warn before leaving a page**.

Example: unsaved form.

19. What is Resolve Guard?

Fetches data **before navigating** to a route.

```
resolve() {  
  return this.service.getData();  
}
```

20. What is CanLoad guard?

Prevents lazy-loaded module from being downloaded.

21. What is CanMatch guard?

Added in Angular 15.

Used for **route matching** before activating or loading a route.

22. Difference between CanActivate & CanLoad?

CanActivate	CanLoad
Route loads but may be blocked	Route won't load at all
Controls navigation	Controls module loading

Angular Interview Q&A Deep Dive

23. How to redirect routes?

```
{ path: '', redirectTo: 'home', pathMatch: 'full' }
```

24. What is pathMatch?

- full: URL must match exactly
 - prefix: URL starts with prefix
-

25. What is wildcard route?

```
{ path: '**', component: PageNotFoundComponent }
```

26. Difference between Absolute & Relative navigation?

Absolute:

```
this.router.navigate(['/admin']);
```

Relative:

```
this.router.navigate(['details'], { relativeTo: this.route });
```

27. What is routerLinkActive?

Adds a CSS class when the route is active.

```
<a routerLink="/home" routerLinkActive="active">Home</a>
```

28. What is Fragment navigation?

```
<a [routerLink]="['/help']" fragment="top">Go Top</a>
```

29. What is Route Reuse Strategy?

Used to **cache components** and reuse them instead of destroying/recreating.

Useful in dashboards, tables, tabs etc.

30. What is Preloading in Angular?

Loads lazy modules **in the background** after app startup.

Angular Interview Q&A Deep Dive

31. Types of preloading strategies?

1. NoPreloading
 2. PreloadAllModules
 3. Custom Preloading Strategies
-

32. How to enable PreloadAllModules?

```
RouterModule.forRoot(routes, {  
  preloadingStrategy: PreloadAllModules  
})
```

33. What is the difference between Hash Routing & HTML5 Routing?

Hash Routing	HTML5 Routing
Uses /#/home	Uses /home
Compatible with all servers	Needs server config
No refresh issues	Refresh may cause 404

34. What is RouterLinkWithHref?

An attribute directive for anchor tags, but combined with routerLink for SPA-style navigation using an <a> tag.

35. How to reload the same route?

```
this.router.navigate([], {  
  relativeTo: this.route,  
  queryParamsHandling: 'merge'  
});
```

OR

Use:

```
onSameUrlNavigation: 'reload'
```

Angular Interview Q&A Deep Dive

36. How to load component dynamically using routing?

Use *loadComponent* (Angular 15+):

```
{  
  path: 'contact',  
  loadComponent: () => import('./contact.component').then(m => m.ContactComponent)  
}
```

37. What is Standalone Routing?

Introduced in Angular 15.

```
import { provideRouter } from '@angular/router';
```

```
bootstrapApplication(AppComponent, {  
  providers: [provideRouter(routes)]  
});
```

38. What is initialNavigation?

Controls how router performs the first navigation (SSR related).

39. How to cancel navigation?

Use **CanDeactivate guard** OR return false from CanActivate.

40. What is ActivatedRouteSnapshot?

Represents the **state of the route at the moment it was activated**.

41. What is RouterStateSnapshot?

Represents the **entire router state** at activation time.

42. How to get previous route?

Use a global Router Event listener.

Angular Interview Q&A Deep Dive

43. What is NavigationEnd event?

Emitted when navigation is fully completed.

44. What is NavigationError?

Event triggered when navigation fails.

45. What is TitleStrategy? (Angular 14+)

Used to **set page title dynamically** from routing.

Angular Routing – Real Project Architecture Questions & Answers

1. How do you structure routing in a large-scale Angular application?

Answer:

In real projects, routing is **modularized**:

- **AppRoutingModule**
Only defines the top-level routes + lazy loading.
- **Feature Routing Modules**
Each domain module (e.g., User, Admin, Orders) has its own routing module.
- **Shared Module**
Contains shared components used across multiple modules, but **no routing**.

Example structure:

/app

 /admin

 admin.module.ts

 admin-routing.module.ts

 /user

 user.module.ts

 user-routing.module.ts

 /shared

 shared.module.ts

app-routing.module.ts

This keeps routing **clean, scalable, and maintainable**.

Angular Interview Q&A Deep Dive

2. How do you decide which module should be lazy loaded?

Answer:

Lazy load modules that are:

- Not used during the initial load
- Large or rarely accessed features
- Admin sections
- Reports, analytics, configuration pages

This improves **performance** and reduces **bundle size**, especially for enterprise apps.

3. How do you handle authentication and authorization using routing?

Answer:

Use a combination of **guards**:

- CanActivate → Allow/deny access
- CanActivateChild → Protect child routes
- CanLoad or CanMatch → Prevent loading lazy modules
- Resolve → Fetch permissions

Example flow:

1. User logs in → fetch JWT + permissions
2. Store permissions in service or storage
3. Guard checks permission and decides

```
canActivate(): boolean {  
  return this.auth.hasPermission('ADMIN');  
}
```

This ensures **role-based routing control**.

4. How do you manage dynamic menus based on roles?

Answer:

Use **route metadata** and filter based on user permissions.

```
{  
  path: 'admin',  
  component: AdminComponent,
```

Angular Interview Q&A Deep Dive

```
data: { requiredRole: 'ADMIN' }  
}
```

Menu generation service:

```
this.routes.filter(r => userRoles.includes(r.data.requiredRole));
```

This avoids hardcoding menu logic.

5. How do you handle wildcard or fallback routes in a production app?

Answer:

Use:

- `**` → Page not found
- Route ordering → Wildcard at last
- Separate 403 and 404 pages

```
{ path: 'forbidden', component: ForbiddenComponent }
```

```
{ path: '**', component: PageNotFoundComponent }
```

6. How do you protect API calls based on routing?

Answer:

Routing guards prevent unauthorized views, but **interceptors** secure API calls:

```
intercept(req) {  
  const authReq = req.clone({  
    headers: req.headers.set('Authorization', `Bearer ${token}`)  
  });  
  return next.handle(authReq);  
}
```

Needed because routing protection is **client-side** only.

7. How do you implement a "Leave page with unsaved changes" warning?

Answer:

Use **CanDeactivate** guard.

```
canDeactivate(component: FormComponent) {  
  return component.isFormDirty ? confirm("Discard changes?") : true;  
}
```

Angular Interview Q&A Deep Dive

```
}
```

Used in forms-heavy enterprise apps.

8. How do you share data between routes?

Methods:**

1. **Route params**
2. **Query params**
3. **State navigation**
4. **Services with BehaviorSubject**
5. **Resolver**

Resolvers are used when data must be present **before loading** a route.

9. How do you preload selected lazy modules?

Answer:

Use a **custom preloading strategy**:

```
preload(route, load) {  
  return route.data?.preload ? load() : EMPTY;  
}
```

Route config:

```
{ path: 'reports', loadChildren: ..., data: { preload: true } }
```

This speeds up frequently accessed but lazy-loaded modules.

10. How do you handle route-based page titles in a real-life app?

Answer:

Use **TitleStrategy** (Angular 14+).

```
{  
  path: 'dashboard',  
  component: DashboardComponent,  
  title: 'Admin Dashboard'  
}
```

Custom strategy:

Angular Interview Q&A Deep Dive

```
override updateTitle(snapshot) {  
  const title = this.buildTitle(snapshot);  
  this.titleService.setTitle('My App - ' + title);  
}
```

Great for **branding and SEO**.

11. How do you optimize routing for mobile performance?

Answer:

- Lazy load >90% of features
- Use custom preloading
- Load components via **loadComponent()** (Angular 15+)
- Avoid giant modules
- Cache heavy routes using RouteReuseStrategy

Mobile users get faster load times.

12. How do you handle deep nested routes (3–4 levels)?

Answer:

Use:

- **children arrays**
- **secondary outlets** if needed
- **clear folder structure**
- **route resolvers** to avoid repeated API calls

Also avoid too many nested router-outlets → bad UX.

13. How do you track previous route?

Answer:

Use a global Router event listener:

```
router.events.subscribe(e => {  
  if (e instanceof NavigationEnd) {  
    this.previous = this.current;  
    this.current = e.url;  
  }  
});
```

Angular Interview Q&A Deep Dive

```
}
```

```
});
```

Used in:

- Back buttons
 - Breadcrumb
 - Audit logs
-

14. How do you implement breadcrumb navigation?

Answer:

Use route metadata with parent-child hierarchy:

```
{  
  path: 'orders',  
  component: OrdersComponent,  
  data: { breadcrumb: 'Orders' }  
}
```

Breadcrumb service reads metadata from `ActivatedRouteSnapshot`.

15. How do you handle multi-language routes?

Example URLs:

- `/en/products`
- `/fr/products`

Solution:

```
{ path: ':lang', children: [...] }
```

OR use **middleware rewrite** (recommended).

16. How do you secure admin routes at build level?

Answer:

Use:

- Guards → protect activation
 - CanLoad → restrict module download
 - Server-side rewrite rules → restrict access
-

Angular Interview Q&A Deep Dive

- Backend role verification → validate JWT per route

Securing only frontend is NOT enough.

17. How do you handle route-based animations?

Answer:

```
<main [@routeAnimation]="o.isActive ? o.activatedRoute : "">  
  <router-outlet #o="outlet"></router-outlet>  
</main>
```

Animations are triggered based on route change events.

18. How do you debug routing in large apps?

Best practices:

- ✓ Use enableTracing: true
 - ✓ Use router events log
 - ✓ Inspect ActivatedRoute tree
 - ✓ Use Chrome → Routing tree in Angular DevTools
-

19. How do you migrate a large old Angular app to standalone routing?

Steps:

1. Convert modules → standalone components
 2. Replace module routing with provideRouter
 3. Use loadComponent() and loadChildren()
 4. Migrate guards & resolvers
 5. Test all lazy-loaded routes
-

20. What routing problems occur in production and how to fix them?

Common issues:

Issue	Fix
404 on refresh	Configure server rewrite to index.html
Double navigation	Use onSameUrlNavigation: 'reload'
Breadcrumb incorrect	Fix nested route metadata

Angular Interview Q&A Deep Dive

Lazy modules not loading	Incorrect path or CanLoad blocking
Route flickering	Use resolvers

Angular Routing — Real Coding Tasks Asked in Interviews

These are practical, hands-on tasks used by MNCs and product-based companies.

1. Create a Lazy-Loaded Module with Routing

Task

Create a module named products and lazy load it using Angular routing. Inside it, create two routes:

- /products/list
- /products/details/:id

Solution

app.routes.ts (Angular 17+)

```
export const routes: Routes = [  
  {  
    path: 'products',  
    loadChildren: () => import('./products/products.routes')  
      .then(m => m.PRODUCT_ROUTES)  
  }  
];
```

products.routes.ts

```
import { Routes } from '@angular/router';  
  
import { ProductListComponent } from './list/list.component';  
import { ProductDetailsComponent } from './details/details.component';  
  
export const PRODUCT_ROUTES: Routes = [  
  { path: 'list', component: ProductListComponent },  
  { path: 'details/:id', component: ProductDetailsComponent }  
];
```

Angular Interview Q&A Deep Dive

✅ 2. Create a Route Guard to Prevent Access Without Login

Task

Implement a guard that:

- Redirects to /login
- Blocks route /dashboard if the user is not logged in

Solution

auth.guard.ts

```
import { CanActivateFn, Router } from '@angular/router';
```

```
import { inject } from '@angular/core';
```

```
export const authGuard: CanActivateFn = () => {
```

```
  const router = inject(Router);
```

```
  const isLoggedIn = !!localStorage.getItem('token');
```

```
  if (!isLoggedIn) {
```

```
    router.navigate(['/login']);
```

```
    return false;
```

```
  }
```

```
  return true;
```

```
};
```

route config

```
{
```

```
  path: 'dashboard',
```

```
  canActivate: [authGuard],
```

```
  loadComponent: () => import('./dashboard/dashboard.component')
```

```
}
```

✅ 3. Implement CanDeactivate Guard for Unsaved Form

Task

User must be warned before leaving a component if the form is dirty.

Solution

Angular Interview Q&A Deep Dive

form.component.ts

```
export class FormComponent {  
  
  isDirty = true;  
  
  canExit() {  
    return !this.isDirty || confirm("You have unsaved changes!");  
  }  
}
```

can-deactivate.guard.ts

```
import { CanDeactivateFn } from '@angular/router';  
  
export const exitGuard: CanDeactivateFn<any> = (component) => {  
  return component.canExit ? component.canExit() : true;  
};
```

routes

```
{  
  path: 'edit',  
  component: FormComponent,  
  canDeactivate: [exitGuard]  
}
```

4. Build Breadcrumb Navigation from Route Data

Task

Show a breadcrumb like:

Home > Users > Details

Use route metadata to drive breadcrumb.

Solution

routes

```
{  
  path: 'users',  
  data: { breadcrumb: 'Users' },  
}
```

Angular Interview Q&A Deep Dive

```
children: [  
  {  
    path: 'details/:id',  
    data: { breadcrumb: 'Details' },  
    loadComponent: () => import('./details/details.component')  
  }  
]  
}
```

breadcrumb.service.ts

```
import { Injectable } from '@angular/core';  
import { ActivatedRouteSnapshot } from '@angular/router';  
  
@Injectable({ providedIn: 'root' })  
export class BreadcrumbService {  
  getBreadcrumbs(route: ActivatedRouteSnapshot, url = '', breadcrumbs = []) {  
    const label = route.data['breadcrumb'];  
    const path = route.routeConfig?.path;  
  
    if (label) {  
      const nextUrl = `${url}/${path}`;  
      breadcrumbs.push({ label, url: nextUrl });  
      url = nextUrl;  
    }  
  
    if (route.firstChild) {  
      return this.getBreadcrumbs(route.firstChild, url, breadcrumbs);  
    }  
  
    return breadcrumbs;  
  }  
}
```

Angular Interview Q&A Deep Dive

✅ 5. Route Resolver – Load Data Before Component Loads

Task

Fetch user details **before** navigating to /user/:id.

Solution

resolver

```
export const userResolver: ResolveFn<any> = (route) => {  
  const id = route.paramMap.get('id');  
  return fetch(`https://jsonplaceholder.typicode.com/users/${id}`)  
    .then(res => res.json());  
};
```

route

```
{  
  path: 'user/:id',  
  resolve: { user: userResolver },  
  loadComponent: () => import('./user/user.component')  
}
```

component

```
constructor(private route: ActivatedRoute) {}
```

```
ngOnInit() {  
  this.user = this.route.snapshot.data['user'];  
}
```

✅ 6. Create a Feature Module with Child Routes

Task

Create an admin module with:

- /admin/dashboard
- /admin/users
- /admin/settings

Solution

Angular Interview Q&A Deep Dive

admin.routes.ts

```
export const ADMIN_ROUTES: Routes = [  
  { path: 'dashboard', loadComponent: () => import('./dashboard.component') },  
  { path: 'users', loadComponent: () => import('./users.component') },  
  { path: 'settings', loadComponent: () => import('./settings.component') }  
];
```

app.routes.ts

```
{  
  path: 'admin',  
  loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)  
}
```

✅ 7. Create Route Animations

Task

Fade animation when navigating between pages.

Solution

app.component.html

```
<div [@routeAnimation]="routerOutlet.isActive ? routerOutlet.activatedRoute : "">  
  <router-outlet #routerOutlet="outlet"></router-outlet>  
</div>
```

animation

```
trigger('routeAnimation', [  
  transition('* <=> *', [  
    style({ opacity: 0 }),  
    animate('500ms', style({ opacity: 1 }))  
  ])  
]);
```

✅ 8. Build a Role-Based Route System

Task

Allow:

Angular Interview Q&A Deep Dive

- Admin → All routes
- User → Only /products
- Guest → Only /login

Solution

role.guard.ts

```
export const roleGuard: CanActivateFn = (route) => {  
  const userRole = localStorage.getItem('role');  
  const required = route.data['role'];  
  
  return userRole === required;  
};
```

routes

```
{  
  path: 'admin',  
  canActivate: [roleGuard],  
  data: { role: 'admin' },  
  loadChildren: () => import('./admin/admin.routes')  
}
```

✅ 9. Routing With Secondary Outlet (popup/modal)

Task

Open a modal using router-outlet:

URL example:

/users(view:modal)

Solution

template

```
<router-outlet></router-outlet>  
  
<router-outlet name="popup"></router-outlet>
```

routes

```
{  
  path: 'users',
```


Angular Interview Q&A Deep Dive

```
    component: UsersListComponent
  },
  {
    path: 'edit/:id',
    component: UserEditComponent,
    outlet: 'popup'
  }
}
```

10. Custom Preloading Strategy

Task

Preload only modules with data: { preload: true }.

Solution

preload.strategy.ts

```
export class CustomPreload implements PreloadingStrategy {
  preload(route, fn) {
    return route.data?.['preload'] ? fn() : EMPTY;
  }
}
```

Angular Interview Q&A Deep Dive

Forms (Template & Reactive) Module

1. What are Angular forms?

Angular forms allow you to **capture user input, validate, and handle data** in a structured way.

2. Types of Angular forms

1. **Template-driven forms** → Defined mostly in template using ngModel
 2. **Reactive forms** → Defined in TypeScript using FormControl and FormGroup
-

3. Key differences: Template vs Reactive

Feature	Template-driven	Reactive
Form model location	Template	Component (TS)
Validation	Template	TS/Reactive
Complexity	Simple	Scalable/Advanced
Change detection	NgModel	Observables
Unit testing	Hard	Easy

4. Which module is needed for Template-driven forms?

FormsModule (import in AppModule)

```
import { FormsModule } from '@angular/forms';
```

5. Which module is needed for Reactive forms?

ReactiveFormsModule

```
import { ReactiveFormsModule } from '@angular/forms';
```

6. How to create a Template-driven form?

```
<form #myForm="ngForm" (ngSubmit)="submit(myForm)">
  <input name="username" ngModel required>
  <button type="submit">Submit</button>
</form>
```

Angular Interview Q&A Deep Dive

7. How to create a Reactive form?

```
form = new FormGroup({
  username: new FormControl("", Validators.required)
});

<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="username">
  <button type="submit">Submit</button>
</form>
```

8. How to access form values?

- **Template-driven:** myForm.value
 - **Reactive:** this.form.value
-

9. How to reset forms?

- myForm.reset() → Template
 - this.form.reset() → Reactive
-

10. How to patch a form value in Reactive Forms?

```
this.form.patchValue({ username: 'Rahul' });
```

11. Difference between setValue() and patchValue()

- setValue() → Updates **all** controls, else error
 - patchValue() → Updates **specific** controls
-

12. How to handle form validation in Template-driven?

Use **validators** like **required**, **minlength**, **pattern**:

```
<input name="email" ngModel required email>
```

Access errors:

```
<div *ngIf="myForm.controls.email?.errors?.required">Required</div>
```

Angular Interview Q&A Deep Dive

13. How to handle validation in Reactive forms?

Use Validators in TS:

```
username: new FormControl('', [Validators.required, Validators.minLength(3)])
```

Access errors:

```
this.form.controls['username'].errors
```

14. How to show real-time validation messages?

- Template-driven: `myForm.controls.name.invalid` && `myForm.controls.name.touched`
 - Reactive: `form.get('controlName')?.invalid` && `form.get('controlName')?.touched`
-

15. How to use custom validators?

```
function customValidator(control: FormControl) {  
  return control.value.startsWith('A') ? null : { invalidStart: true };  
}
```

16. How to use async validators?

```
username: new FormControl('', null, this.usernameAsyncValidator)
```

Return Promise or Observable with { `errorName: true` } or null.

17. What is FormArray?

Group of **dynamic FormControl**s in Reactive Forms.

```
this.form = new FormGroup({  
  phones: new FormArray([new FormControl('')])  
});
```

Add dynamically:

```
(this.form.get('phones') as FormArray).push(new FormControl('123'));
```

18. How to use ngModel with select dropdown?

```
<select [(ngModel)]="selectedValue" name="country">  
  <option *ngFor="let c of countries" [value]="c">{{c}}</option>  
</select>
```

Angular Interview Q&A Deep Dive

19. How to bind multiple checkboxes?

Template-driven:

```
<input type="checkbox" name="hobbies" ngModel [value]="Reading">
```

Reactive:

```
new FormControl([]) // array of selected items
```

20. How to dynamically add/remove controls in Reactive Forms?

```
this.form.addControl('newCtrl', new FormControl(''));
```

```
this.form.removeControl('newCtrl');
```

21. Difference between touched, dirty, pristine

Property Meaning

touched User has focused and blurred

dirty User has changed value

pristine Not changed yet

22. How to disable/enable controls dynamically?

```
this.form.get('username')?.disable();
```

```
this.form.get('username')?.enable();
```

23. How to listen to form changes?

- **Reactive:** `this.form.valueChanges.subscribe(val => ...)`
 - **Template-driven:** Use `(ngModelChange)` or `RxJS`
-

24. How to check form validity?

```
this.form.valid // true/false
```

25. How to submit form programmatically?

- Template-driven: `myForm.ngSubmit.emit()`
-

Angular Interview Q&A Deep Dive

- Reactive: this.submit() method bound to (ngSubmit)

26. How to disable submit button until form is valid?

```
<button [disabled]="form.invalid">Submit</button>
```

27. How to create nested forms?

```
new FormGroup({  
  address: new FormGroup({  
    city: new FormControl(""),  
    zip: new FormControl("")  
  })  
});
```

28. What is the difference between Template-driven & Reactive Forms validation approach?

- Template-driven → validation in **HTML template**
- Reactive → validation in **TypeScript code**

29. How to handle cross-field validation?

Example: Password & Confirm Password must match.

```
this.form = new FormGroup({  
  password: new FormControl(""),  
  confirmPassword: new FormControl("")  
}, { validators: this.passwordMatchValidator });
```

30. How to implement conditional validation?

```
if (condition) this.form.get('phone')?.setValidators(Validators.required);  
else this.form.get('phone')?.clearValidators();
```

Angular Interview Q&A Deep Dive

31. What is the difference between ngForm and FormGroup?

ngForm	FormGroup
Template-driven	Reactive
Manages controls automatically	Controls defined in TS
Less scalable	Scalable

32. How to patch nested FormGroup values?

```
this.form.get('address')?.patchValue({ city: 'Mumbai' });
```

33. How to create a reusable form component?

Use **@Input()** for FormGroup/FormControl and **@Output()** for submit event.

34. How to handle form reset to initial value?

```
this.form.reset({ username: 'Default' });
```

35. How to handle dynamic validators on a control?

```
this.form.get('email')?.setValidators([Validators.required, Validators.email]);  
this.form.get('email')?.updateValueAndValidity();
```

36. How to use FormBuilder service?

```
constructor(private fb: FormBuilder) {}  
  
this.form = this.fb.group({  
  username: ['', Validators.required],  
  email: ['', [Validators.required, Validators.email]]  
});
```

37. How to handle checkbox groups in Reactive forms?

```
new FormArray([new FormControl(false), new FormControl(false)])  
  
Use map to get selected values.
```

Angular Interview Q&A Deep Dive

38. How to use async validators with Reactive forms?

```
new FormControl("", null, this.uniqueUsernameValidator.bind(this))
```

Must return Observable<ValidationErrors | null>.

39. Difference between valueChanges and statusChanges

valueChanges

statusChanges

Emits when value changes Emits when validation status changes

Can use to auto-save

Can use to enable/disable buttons

40. How to prevent form submission if invalid?

```
<form (ngSubmit)="submit()" #myForm="ngForm">
  <button [disabled]="myForm.invalid">Submit</button>
</form>
```

41. How to validate pattern in template-driven forms?

```
<input type="text" name="phone" ngModel pattern="[0-9]{10}">
```

42. How to pre-fill form with existing data?

- Template-driven: Use [ngModel]="user.username"
 - Reactive: Use patchValue() or setValue()
-

43. How to use async pipe with Reactive forms?

For example, populating a dropdown from API:

```
<select [formControl]="cityControl">
  <option *ngFor="let city of cities$ | async" [value]="city.id">{{city.name}}</option>
</select>
```

44. How to disable submit button until valid?

```
<button type="submit" [disabled]="form.invalid">Submit</button>
```

Angular Interview Q&A Deep Dive

45. How to create dynamic forms (dynamic fields)?

- Use FormArray to add/remove controls dynamically.
 - Example: Add multiple phone numbers.
-

46. How to implement a custom error message component for forms?

- Create component form-error with input control
 - Show error messages dynamically based on control.errors
-

47. How to detect if a field is dirty or touched?

control.dirty

control.touched

48. How to validate at form level (cross-field)?

Use validator function on FormGroup.

49. How to integrate Reactive forms with Material UI?

- Bind FormControl to Material input
- Use mat-error for validation messages

```
<mat-form-field>
```

```
  <input matInput formControlName="email">
```

```
  <mat-error *ngIf="form.get('email')?.invalid">Invalid email</mat-error>
```

```
</mat-form-field>
```

50. How to unit test Reactive forms?

- Test FormGroup creation
- Test validation rules
- Test valueChanges

```
expect(form.valid).toBeFalsy();
```

```
form.get('username')?.setValue('Rahul');
```

```
expect(form.valid).toBeTruthy();
```

Angular Interview Q&A Deep Dive

RxJS Module:

1. What is RxJS?

RxJS (Reactive Extensions for JavaScript) is a **library for reactive programming using Observables**, allowing you to **work with asynchronous data streams**.

2. What is an Observable?

An Observable represents a **stream of data** over time, which can be **subscribed to**.

```
import { Observable } from 'rxjs';

const obs = new Observable(subscriber => {

  subscriber.next(1);

  subscriber.next(2);

  subscriber.complete();

});
```

3. What is an Observer?

An Observer is an object that **receives notifications** from an Observable.

```
obs.subscribe({

  next: value => console.log(value),

  error: err => console.error(err),

  complete: () => console.log('Done')

});
```

4. Difference between Observable and Promise

Feature	Observable	Promise
Multiple values	Yes	No
Lazy execution	Yes	No
Cancelable	Yes	No
Operators	Many	Few

5. What is a Subject in RxJS?

A Subject is both **Observable and Observer**. It allows **multicasting** to multiple subscribers.

Angular Interview Q&A Deep Dive

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

subject.subscribe(val => console.log('A:', val));

subject.next(1);
```

6. Difference between Subject, BehaviorSubject, ReplaySubject, AsyncSubject

Type	Key Feature
Subject	No initial value, emits to current subscribers only
BehaviorSubject	Requires initial value, emits latest value to new subscribers
ReplaySubject	Emits last n values to new subscribers
AsyncSubject	Emits only the last value when completed

7. What are Operators in RxJS?

Operators are functions that **transform, filter, or combine Observables**.

- **Pipeable operators:** map, filter, tap, debounceTime
 - **Creation operators:** of, from, interval, timer
 - **Combination operators:** merge, concat, forkJoin, combineLatest
-

8. Difference between map and switchMap

- map → transforms emitted values
- switchMap → maps to inner observable, **cancels previous** if new value comes

```
obs.pipe(switchMap(val => http.get(`/api/${val}`)));
```

9. What is mergeMap vs concatMap?

- mergeMap → runs inner observables in **parallel**
 - concatMap → runs **sequentially**
-

10. What is forkJoin?

Combines multiple observables and emits **final results only when all complete**.

```
forkJoin([obs1, obs2]).subscribe(console.log);
```

Angular Interview Q&A Deep Dive

11. What is combineLatest?

Emits latest values **from all observables** whenever **any observable emits**.

12. What is withLatestFrom?

Combines **source observable** with **latest value** from another observable **when source emits**.

13. What is take, takeUntil, first, last?

- take(n) → take first n values
 - takeUntil(otherObs) → take until otherObs emits
 - first() → first emitted value
 - last() → last emitted value
-

14. How to unsubscribe from Observable?

- Using Subscription:

```
const sub = obs.subscribe();
```

```
sub.unsubscribe();
```

- Using takeUntil pattern in Angular for **automatic cleanup**:

```
destroy$ = new Subject<void>();
```

```
obs.pipe(takeUntil(this.destroy$)).subscribe();
```

15. What is Subject used for in Angular?

- Event emitter between components
 - Shared state management
 - Trigger async actions
-

16. What is BehaviorSubject used for?

- Holds **current value**
- Can be subscribed **anytime** to get latest value

```
const bSub = new BehaviorSubject(0);
```

```
bSub.next(5);
```

```
bSub.subscribe(val => console.log(val)); // prints 5 immediately
```

Angular Interview Q&A Deep Dive

17. What is the difference between Observable and Subject?

Observable	Subject
Unicast	Multicast
Lazy	Hot by default
Cannot manually emit	Can manually emit values using next()

18. How to handle HTTP requests in Angular using RxJS?

- Angular HttpClient returns **Observables**.

```
this.http.get('/api/users').subscribe(data => console.log(data));
```

19. What is the difference between subscribe and async pipe?

- subscribe → manually subscribe/unsubscribe
- async → template binding, automatically unsubscribes

```
<div *ngFor="let user of users$ | async">{{user.name}}</div>
```

20. What is the difference between cold and hot observables?

Cold Observable	Hot Observable
Starts execution on subscribe	Already producing values
Each subscriber gets its own execution	All subscribers share execution
Example: HttpClient	Example: Subject, EventEmitter

21. What is debounceTime and where is it used?

- Delays emitting values **until user stops typing**
- Used in **search boxes**:

```
this.searchControl.valueChanges.pipe(debounceTime(300)).subscribe(...);
```

22. What is distinctUntilChanged?

Prevents emitting duplicate consecutive values.

Angular Interview Q&A Deep Dive

23. Difference between switchMap, mergeMap, exhaustMap

Operator	Behavior
----------	----------

switchMap	Cancels previous inner observable
-----------	-----------------------------------

mergeMap	Runs all inner observables in parallel
----------	--

concatMap	Queues inner observables
-----------	--------------------------

exhaustMap	Ignores new inner observable until current completes
------------	--

24. What is retry in RxJS?

Retries a failing observable **N times** before erroring.

25. What is catchError?

Catches errors and allows **fallback logic**.

```
this.http.get('/api/users')
```

```
.pipe(catchError(err => of([])))
```

```
.subscribe(data => console.log(data));
```

26. Difference between retry and catchError

- `retry` → repeats the observable if it fails
 - `catchError` → handles the error, can return fallback observable
-

27. What is shareReplay?

Shares a **single subscription** among multiple subscribers and replays **last N values**.

28. What is throttleTime?

Prevents **frequent emissions**, useful in scrolling events or button clicks.

29. What is combineLatest vs zip?

Operator	Emits when	Output
combineLatest	Any observable emits	Latest values of all
zip	All observables emit	Values paired by index

Angular Interview Q&A Deep Dive

30. How to cancel HTTP requests using RxJS?

Use **takeUntil** with **Subject** or **switchMap** to cancel previous requests.

31. What is timer and interval?

- `timer(1000)` → emits **once** after 1 sec
 - `interval(1000)` → emits **every 1 sec**
-

32. What is fromEvent?

Creates an observable from **DOM events**.

```
fromEvent(document, 'click').subscribe(() => console.log('Clicked'));
```

33. What is merge, concat, race?

- `merge` → merge multiple observables in parallel
 - `concat` → sequentially one after another
 - `race` → emits from **first observable** to emit
-

34. How to debug RxJS streams?

Use **tap** operator:

```
obs.pipe(tap(value => console.log(value))).subscribe();
```

35. How to implement caching using RxJS?

Use **shareReplay(1)**:

```
this.user$ = this.http.get('/api/users').pipe(shareReplay(1));
```

36. Difference between of and from

- `of(1,2,3)` → emits **arguments as values**
 - `from([1,2,3])` → emits **array elements one by one**
-

37. What is the difference between subscribe and pipe?

- `pipe` → transforms observable using operators
-

Angular Interview Q&A Deep Dive

- subscribe → triggers execution
-

38. How to implement search with debounce and switchMap?

this.searchControl.valueChanges

```
.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.http.get(`/api/search?q=${term}`))
).subscribe(results => this.results = results);
```

39. How to handle multiple API calls in parallel?

Use forkJoin:

```
forkJoin([this.http.get('/api/users'), this.http.get('/api/posts')])
  .subscribe(([users, posts]) => console.log(users, posts));
```

40. How to combine latest values from multiple sources?

Use combineLatest:

```
combineLatest([obs1$, obs2$]).subscribe(([val1, val2]) => console.log(val1, val2));
```

41. Difference between switchMap vs exhaustMap in login forms

- switchMap → cancels previous request if user submits again quickly
 - exhaustMap → ignores new submissions until current completes
-

42. How to use startWith in RxJS?

Emits **initial value** before source emits.

```
this.obs.pipe(startWith(0)).subscribe();
```

43. Difference between scan and reduce

- scan → accumulates **values over time**, emits **every step**
 - reduce → accumulates, emits **only final value**
-

Angular Interview Q&A Deep Dive

44. How to throttle clicks using RxJS?

```
fromEvent(button, 'click').pipe(throttleTime(1000)).subscribe();
```

45. Difference between multicast and share

- multicast → uses **Subject** to share single subscription
 - share → automatically multicast and ref-count
-

46. How to handle retry with delay in HTTP requests?

```
this.http.get('/api/data')  
  .pipe(retryWhen(errors => errors.pipe(delay(1000), take(3))))  
  .subscribe();
```

47. How to cancel long-running HTTP requests on component destroy?

Use takeUntil pattern:

```
private destroy$ = new Subject<void>();  
obs$.pipe(takeUntil(this.destroy$)).subscribe();  
ngOnDestroy() { this.destroy$.next(); this.destroy$.complete(); }
```

48. How to combine valueChanges of multiple FormControls?

```
combineLatest([  
  this.form.get('firstName')!.valueChanges,  
  this.form.get('lastName')!.valueChanges  
]).subscribe(([first, last]) => console.log(first, last));
```

49. How to handle errors in a stream and continue?

Use catchError and return fallback observable:

```
obs$.pipe(catchError(err => of('default'))).subscribe();
```

50. How to handle event-driven architecture using RxJS?

- Use Subject or BehaviorSubject to emit **global events**
 - Components subscribe to these streams for **reactive updates**
-

Angular Interview Q&A Deep Dive

```
this.eventService.userLoggedIn$.subscribe(user => {...});
```

51. Difference between Observable.create and new Observable()

- Observable.create() → legacy way, deprecated
 - new Observable() → recommended
-

52. What is the difference between defer and of?

- of() → creates observable **immediately**
 - defer() → creates observable **on subscription**
-

53. How to unsubscribe automatically with async pipe?

- async pipe **handles unsubscribe** automatically.

```
<div *ngFor="let item of items$ | async">{{item}}</div>
```

Angular Interview Q&A Deep Dive

Directives Module:

1. What is a directive in Angular?

A directive is a **class that adds behavior to elements** in your Angular application.

2. Types of directives

1. **Component Directives** → Components themselves are directives with templates.
 2. **Attribute Directives** → Change **appearance or behavior** of elements (e.g., `ngStyle`, `ngClass`).
 3. **Structural Directives** → Change **DOM structure** (e.g., `*ngIf`, `*ngFor`, `*ngSwitch`).
-

3. Difference between Component and Directive

Feature	Component	Directive
Template	Has its own	Uses host element
Selector	Element	Attribute or element
Purpose	UI	Behavior/structure

4. What is `*ngIf`?

Structural directive that **adds/removes element** from DOM based on a condition.

```
<div *ngIf="isVisible">Hello</div>
```

5. What is `*ngFor`?

Structural directive that **loops over a collection**.

```
<div *ngFor="let item of items; index as i">{{i}} - {{item}}</div>
```

6. What is `*ngSwitch`?

Structural directive for **switch-case logic** in templates.

```
<div [ngSwitch]="value">
  <p *ngSwitchCase="'A'">A</p>
  <p *ngSwitchCase="'B'">B</p>
  <p *ngSwitchDefault>Default</p>
</div>
```

Angular Interview Q&A Deep Dive

7. What is ngStyle?

Attribute directive to **dynamically apply inline styles**.

```
<div [ngStyle]='{color: color, 'font-size.px': fontSize}'>Text</div>
```

8. What is ngClass?

Attribute directive to **add/remove CSS classes** dynamically.

```
<div [ngClass]='{active: isActive, 'disabled': isDisabled}'>Div</div>
```

9. How to create a custom attribute directive?

```
import { Directive, ElementRef, Renderer2 } from '@angular/core';
```

```
@Directive({ selector: '[appHighlight]' })
```

```
export class HighlightDirective {
```

```
  constructor(private el: ElementRef, private renderer: Renderer2) {
```

```
    this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'yellow');
```

```
  }
```

```
}
```

Usage:

```
<p appHighlight>Highlighted text</p>
```

10. Difference between structural and attribute directive

Feature	Structural	Attribute
DOM changes	Add/remove elements	Modify element behavior/style
Prefix	*	none
Examples	*ngIf, *ngFor	[ngStyle], [ngClass]

11. Can you combine multiple directives on one element?

Yes, multiple directives (attribute or structural) can be applied to a single element.

12. How does Angular differentiate *ngIf from ngIf?

* is **syntactic sugar** for <ng-template>.

Angular Interview Q&A Deep Dive

*ngIf="cond" → <ng-template [ngIf]="cond"><div>...</div></ng-template>

13. What is Renderer2?

Service used in **custom directives/components** to manipulate DOM **safely**.

```
this.renderer.setStyle(el.nativeElement, 'color', 'red');
```

14. What is ElementRef?

Gives **direct access to DOM element** in custom directive/component.

15. How to pass input to a custom directive?

```
@Input() appHighlight: string;
```

```
<p [appHighlight]="red">Text</p>
```

16. How to listen to DOM events in a directive?

```
@HostListener('mouseenter') onMouseEnter() {  
  this.renderer.setStyle(this.el.nativeElement, 'color', 'blue');  
}
```

17. What is HostBinding?

Binds a **property or style** of the host element.

```
@HostBinding('style.color') color = 'red';
```

18. Difference between HostListener and HostBinding

- HostListener → listens to events
 - HostBinding → binds properties/styles/classes
-

19. Can you use multiple structural directives on the same element?

No, only **one structural directive** per element is allowed. Use <ng-container> to combine.

20. What is ng-container?

Wrapper element **not rendered in DOM**, useful to apply multiple structural directives.

Angular Interview Q&A Deep Dive

21. How to create a structural custom directive?

```
import { Directive, TemplateRef, ViewContainerRef, Input } from '@angular/core';
```

```
@Directive({ selector: '[appUnless]' })
```

```
export class UnlessDirective {
```

```
  constructor(private tpl: TemplateRef<any>, private vcr: ViewContainerRef) {}
```

```
  @Input() set appUnless(condition: boolean) {
```

```
    if (!condition) this.vcr.createEmbeddedView(this.tpl);
```

```
    else this.vcr.clear();
```

```
  }
```

```
}
```

Usage:

```
<p *appUnless="isVisible">This shows only if isVisible is false</p>
```

22. What is ngTemplateOutlet?

Directive that **renders a <ng-template> dynamically**.

```
<ng-container *ngTemplateOutlet="tpl"></ng-container>
```

23. How to dynamically create components using directives?

Use **ViewContainerRef**:

```
vcr.createComponent(factory);
```

24. Difference between <ng-template> and <ng-container>

Element	Usage
ng-template	Template for structural directive, rendered later
ng-container	Wrapper, not rendered, used for grouping elements

25. What is trackBy in *ngFor?

Optimizes DOM rendering by **tracking items** by unique ID.

Angular Interview Q&A Deep Dive

```
*ngFor="let item of items; trackBy: trackById"
```

26. Difference between ngIf and [hidden]

- ngIf → removes element from DOM
 - [hidden] → hides element using CSS (display:none), DOM remains
-

27. Can we apply attribute directive on <ng-template>?

No, attribute directives require **actual DOM element**, not <ng-template>.

28. How to pass multiple inputs to a directive?

```
@Input() color: string;
```

```
@Input() fontSize: string;
```

29. How to use Renderer2 to add class dynamically?

```
this.renderer.addClass(this.el.nativeElement, 'my-class');
```

```
this.renderer.removeClass(this.el.nativeElement, 'my-class');
```

30. How to make reusable directives?

- Accept **inputs** for styles/behavior
 - Emit events using @Output if needed
-

31. Can directives inject services?

Yes, inject services via **constructor**:

```
constructor(private renderer: Renderer2, private auth: AuthService) {}
```

32. What is the difference between ngClass and custom directive for styling?

- ngClass → predefined attribute directive
 - Custom directive → reusable logic with inputs/events
-

33. How to apply multiple classes dynamically in custom directive?

```
@Input() classes: string[];
```

Angular Interview Q&A Deep Dive

```
classes.forEach(c => this.renderer.addClass(this.el.nativeElement, c));
```

34. Can you use structural directive on a component?

Yes, Angular supports it as long as **component has a template**.

35. How to detect changes in directive input?

Use ngOnChanges lifecycle hook:

```
ngOnChanges(changes: SimpleChanges) { ... }
```

36. How to pass a template reference to directive?

```
@Input() tplRef: TemplateRef<any>;
```

37. What is NgTemplateOutletContext?

Provides **context data** to the template rendered by ngTemplateOutlet.

38. How to toggle visibility using directive?

Use Renderer2 + HostListener + @Input().

39. How to implement drag-and-drop using directives?

Use HostListener for mousedown, mousemove, mouseup events, and manipulate DOM position.

40. How to detect mouse events in attribute directive?

Use @HostListener('mouseenter') and @HostListener('mouseleave').

41. How to create reusable tooltip directive?

- Input: tooltip text
 - HostListener: show/hide tooltip div dynamically using Renderer2
-

42. How to optimize *ngFor performance?

- Use trackBy
 - Avoid unnecessary re-renders
-

Angular Interview Q&A Deep Dive

- Use OnPush change detection if possible
-

43. Can you combine ngIf and ngFor on same element?

No. Use <ng-container> to separate them.

```
<ng-container *ngIf="isVisible">
  <div *ngFor="let item of items">{{item}}</div>
</ng-container>
```

44. How to implement conditional styling using directive?

```
@HostBinding('style.backgroundColor') bg: string;
@Input() set appHighlight(condition: boolean) { this.bg = condition ? 'yellow' : 'white'; }
```

45. What is the role of ViewContainerRef in structural directives?

- Creates or removes **embedded views** dynamically
 - Used with TemplateRef to manipulate DOM
-

46. Difference between ngClass and [class.className]

- [ngClass] → multiple classes dynamically
 - [class.className] → toggle **single class** dynamically
-

47. How to implement accordion using directives?

- Structural directive to **show/hide content**
 - Attribute directive for **toggle behavior** on click
-

48. How to create reusable validation directives?

- Implement Validator interface
- Provide it in **NG_VALIDATORS**

```
@Directive({
  selector: '[appCustomValidator]',
  providers: [{ provide: NG_VALIDATORS, useExisting: forwardRef(() => CustomValidatorDirective),
    multi: true }]
})
```

Angular Interview Q&A Deep Dive

}}

49. How to detect changes on host element dynamically?

- Use MutationObserver or Renderer2 with HostListener

50. Can we use lifecycle hooks in directives?

Yes, all hooks like ngOnInit, ngOnChanges, ngOnDestroy are available.

51. Difference between structural directive and component

- Component → **has template + logic**, used as **element**
- Structural directive → **manipulates DOM**, does not have its own template

52. How to create a reusable dropdown directive?

- Attribute directive toggles **show/hide menu**
- Can bind **selected value** using @Output()

Angular Interview Q&A Deep Dive

Angular Modules:

1. What is an Angular Module (NgModule)?

An NgModule is a **class annotated with @NgModule** that **organizes components, directives, pipes, and services** into cohesive blocks.

2. What are the main properties of @NgModule?

- declarations → Components, directives, pipes
 - imports → Other modules to use
 - exports → Components/directives/pipes to make available outside
 - providers → Services available for DI
 - bootstrap → Root component to bootstrap application
-

3. Difference between Root Module and Feature Module

Root Module	Feature Module
AppModule, bootstraps app	Lazy-loaded or eager-loaded module
Only one in app	Multiple modules
Can import other modules	Can be imported by root or other modules

4. What is a Shared Module?

A module that contains **common components, directives, pipes** that are used across multiple modules.

```
@NgModule({  
  declarations: [CommonComponent, CommonPipe],  
  exports: [CommonComponent, CommonPipe]  
})  
  
export class SharedModule {}
```

5. What is a Core Module?

A module that contains **singleton services** used **once in the app**, e.g., AuthService, LoggerService.

- Imported **only in AppModule**
 - Avoid multiple instances
-

Angular Interview Q&A Deep Dive

6. Difference between Shared Module and Core Module

Shared Module	Core Module
Reusable components/pipes/directives	Singleton services
Can import in multiple modules	Should import only once (AppModule)

7. How to create a Feature Module?

ng generate module feature/feature-name

8. What is Lazy Loading in Angular?

Lazy Loading is **loading modules only when needed** instead of at startup.

```
{ path: 'feature', loadChildren: () => import('./feature/feature.module').then(m => m.FeatureModule)
}
```

9. Difference between Eager and Lazy Loading

Eager Loading

Loaded at app startup

Increases initial bundle

AppModule imports feature module Router loadChildren used

Lazy Loading

Loaded on demand

Reduces initial bundle

10. How to prevent multiple imports of CoreModule?

```
constructor(@Optional() @SkipSelf() parentModule: CoreModule) {
  if(parentModule) throw new Error('CoreModule already loaded');
}
```

11. What is the difference between declarations and exports?

- declarations → Components, directives, pipes **owned by this module**
- exports → Makes **declared components available to other modules**

12. Can services be declared in declarations?

No. Services are added in providers.

Angular Interview Q&A Deep Dive

13. Can components from one module be used in another module without exporting?

No. Components must be **exported** to be used in another module.

14. What is providers in NgModule?

Specifies **services available for dependency injection** within the module.

15. Can multiple modules provide the same service?

Yes, but it can create **multiple instances**. Use **CoreModule singleton pattern** to avoid duplicates.

16. What is forRoot() pattern?

Used to provide **singleton services** in modules:

```
@NgModule({...})
export class AuthModule {
  static forRoot(): ModuleWithProviders<AuthModule> {
    return { ngModule: AuthModule, providers: [AuthService] };
  }
}
```

17. How to import FormsModule and ReactiveFormsModule in feature module?

```
imports: [FormsModule, ReactiveFormsModule]
```

18. How to create a SharedModule?

```
@NgModule({
  declarations: [CommonComponent, CommonPipe],
  exports: [CommonComponent, CommonPipe],
  imports: [CommonModule]
})
export class SharedModule {}
```

19. Difference between CommonModule and BrowserModule

- BrowserModule → Required in **AppModule only**, provides ngIf, ngFor

Angular Interview Q&A Deep Dive

- CommonModule → Used in **feature modules**, provides ngIf, ngFor
-

20. Can you import AppModule into a feature module?

No. This causes a **circular dependency**.

21. What is ModuleWithProviders?

A **pattern for configuring services** in shared or singleton modules using forRoot().

22. How to lazy load a feature module with its own routing?

- Create feature module and routing module
- Use loadChildren in AppRoutingModuleModule

```
{ path: 'products', loadChildren: () => import('./products/products.module').then(m => m.ProductsModule) }
```

23. Difference between NgModule and JavaScript module

- NgModule → Angular **metadata-driven module**, used for DI, compilation
 - JS module → Standard ES module (export/import)
-

24. What is a route-level module?

A feature module that **declares components and routes** specifically for a feature.

25. Can a module import itself?

No, this causes **circular dependency error**.

26. How to share components between feature modules?

- Create **SharedModule**
 - Export components in SharedModule
 - Import SharedModule in feature modules
-

27. How to lazy load with PreloadingStrategy?

```
RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules });
```

Angular Interview Q&A Deep Dive

28. Difference between eagerly and lazily loaded services

- Eager → Singleton from root injector
 - Lazy → New instance created **per lazy module**
-

29. What is a modular architecture in Angular?

- Divide app into **RootModule + FeatureModules + Shared/Core modules**
 - Improves **maintainability and performance**
-

30. Can a component be declared in multiple modules?

No. Components must be declared in **one module only**. Use exports to share.

31. Difference between imports and exports in NgModule

- imports → bring modules/components **into this module**
 - exports → make **this module components available to others**
-

32. How to create lazy-loaded route guards?

Use canLoad guard in route:

```
{ path: 'admin', loadChildren: ..., canLoad: [AuthGuard] }
```

33. How to structure Angular app with modules?

- app/ → AppModule
 - app/core/ → CoreModule
 - app/shared/ → SharedModule
 - app/features/ → Feature modules
 - app/app-routing.module.ts → Routing
-

34. How to lazy load multiple feature modules?

const routes: Routes = [

```
{ path: 'users', loadChildren: () => import('./users/users.module').then(m => m.UsersModule) },
```

Angular Interview Q&A Deep Dive

```
{ path: 'products', loadChildren: () => import('./products/products.module').then(m => m.ProductsModule) }  
];
```

35. What is providers scope in modules?

- Root module → singleton throughout app
 - Feature module → service instance limited to module unless provided in root
-

36. Can we import SharedModule into multiple feature modules?

Yes, SharedModule is **meant to be imported in multiple modules**.

37. Difference between LazyLoadedModule and EagerLoadedModule services

- LazyLoadedModule → **new instance**
 - EagerLoadedModule → singleton shared with root
-

38. How to prevent multiple instances of singleton service in lazy modules?

- Provide service in **CoreModule only**
 - Use forRoot() pattern for shared modules
-

39. Can NgModule be lazy-loaded multiple times?

Yes, but **services will get new instances** unless in CoreModule.

40. How to import third-party modules into feature module?

```
imports: [CommonModule, FormsModule, NgxBootstrapModule]
```

41. How to create a routing module for feature module?

```
@NgModule({  
  imports: [RouterModule.forChild(routes)],  
  exports: [RouterModule]  
})  
  
export class FeatureRoutingModule {}
```

Angular Interview Q&A Deep Dive

42. Difference between `forRoot()` and `forChild()` in `RouterModule`

- `forRoot()` → `AppModule`, sets up router **singleton**
 - `forChild()` → Feature module, adds routes without re-initializing router
-

43. Can `NgModule` be lazy loaded without routing?

No. Lazy loading requires **route-based entry** using `loadChildren`.

44. How to implement modular lazy-loaded app structure?

- `AppModule` + `CoreModule` + `SharedModule` + `FeatureModules`
 - Use Router `loadChildren` for lazy modules
-

45. Can a module import a lazy-loaded module?

No. Lazy-loaded modules are **only loaded by router**.

46. Difference between `AppModule` and `BrowserModule` import

- `AppModule` → main Angular module
 - `BrowserModule` → imports **common directives**, used **once**
-

47. Can `SharedModule` provide services?

Technically yes, but **not recommended**. Use `CoreModule` for **singleton services**.

48. How to handle circular dependency in modules?

- Use **barrel files** carefully
 - Avoid importing **`AppModule` in feature modules**
 - Lazy-load feature modules when possible
-

49. How to create a module with lazy-loaded components?

- `FeatureModule` with `RouterModule.forChild(routes)`
 - Define routes and lazy-load via `AppRoutingModule`
-

Angular Interview Q&A Deep Dive

50. How to split large Angular app into multiple modules?

- Identify features → create **FeatureModule**
 - Identify reusable components → **SharedModule**
 - Singleton services → **CoreModule**
 - Implement **lazy-loading** for feature modules
-

51. Best practices for Angular Modules

- One module per **feature**
- CoreModule → singleton services
- SharedModule → reusable components, directives, pipes
- Avoid importing AppModule in other modules
- Use lazy loading for performance